

ASP.NET Core MVC - Validation

Data Validation in ASP.NET Core

Data validation is the process of ensuring that user input is clean, correct, and useful.

What is Validation?

Validation is the process of **checking whether user input is correct, complete, and safe** before it is processed or stored in the system.

In web applications, users input data through forms (e.g., registration, login, orders). Without validation, this data may:

- Be incomplete (missing required fields)
- Be incorrect (wrong format)
- Be harmful (malicious input like scripts)

Therefore, validation ensures:

- **Data integrity** (correct data is stored)
 - **Security** (prevents attacks)
 - **User experience** (guides users to correct mistakes)
-

Why is Validation Important?

1. Prevents Invalid Data

Example:

- Age = "abc" ✖
- Email = "hello.com" ✖

2. Improves User Experience

- Users immediately know what they did wrong
- Reduces frustration

3. Enhances Security

- Prevents SQL Injection, script attacks, etc.

4. Ensures System Reliability

- Clean and accurate database records
-

TYPES OF VALIDATION (DATA ANNOTATIONS)

In ASP.NET Core, validation is commonly done using **Data Annotations** placed in the model.

1. Required Validation

Ensures that a field is **not empty or null**

```
[Required(ErrorMessage = "Name is required")]  
public string Name { get; set; }
```

If user leaves the field blank → error message appears

2. String Length Validation

Controls the **minimum and maximum number of characters**

```
[StringLength(50, MinimumLength = 5)]  
public string Username { get; set; }
```

Username must be between 5–50 characters

3. Range Validation

Used for numeric values within a **specific range**

```
[Range(18, 60, ErrorMessage = "Age must be between 18 and 60")]  
public int Age { get; set; }
```

Prevents unrealistic values

✓ 4. Regular Expression Validation

Validates input using a **pattern**

```
[RegularExpression(@"^[a-zA-Z]+$", ErrorMessage = "Only letters allowed")]  
public string FirstName { get; set; }
```

Common uses:

- Email
 - Phone number
 - Password format
-

✓ 5. Email Validation

```
[EmailAddress(ErrorMessage = "Invalid email format")]  
public string Email { get; set; }
```

Built-in validator for email format

✓ 6. Compare Validation

Used to compare two fields (e.g., password confirmation)

```
[Compare("Password", ErrorMessage = "Passwords do not match")]
public string ConfirmPassword { get; set; }
```

✓ 7. Custom Error Messages

You can customize feedback:

```
[Required(ErrorMessage = "Please enter your full name")]
```

Makes validation more user-friendly

SERVER-SIDE VALIDATION

Definition

Server-side validation happens **after the form is submitted**, and the data is sent to the server.

How It Works

1. User fills out form
2. Clicks Submit
3. Data is sent to server
4. Server checks validation rules
5. If invalid → returns errors
6. If valid → processes data

Example

Model:

```
public class UserModel
{
    [Required]
    public string Name { get; set; }
}
```

Controller:

```
[HttpPost]
public IActionResult Register(UserModel model)
{
    if (ModelState.IsValid)
    {
        return View("Success");
    }
}
```

```
    return View(model);  
}
```

Key Concept: ModelState

- `ModelState.IsValid` checks if all validations passed
- If false → errors exist

Advantages

- More **secure**
- Cannot be bypassed easily
- Required for all applications

Disadvantages

- Slower (requires server request)
- Less interactive

CLIENT-SIDE VALIDATION

Definition

Validation that happens **in the browser before sending data to the server**

How It Works

- Uses HTML5 or JavaScript
- Prevents submission if input is invalid

Example (HTML5)

```
<input type="text" required />  
<input type="email" />
```

Browser automatically validates

Example (ASP.NET Core Tag Helpers)

```
<input asp-for="Name" class="form-control" />  
<span asp-validation-for="Name" class="text-danger"></span>
```

Displays validation message instantly

Advantages

- Fast feedback
 - Better user experience
 - Reduces server load
-

Disadvantages

- Can be bypassed (user disables JavaScript)
 - That's why server-side validation is still required
-

UNOBTRUSIVE JAVASCRIPT VALIDATION

Definition

A technique where validation rules are written in HTML attributes and automatically handled using JavaScript libraries like:

jQuery

What Does "Unobtrusive" Mean?

- No inline JavaScript code
 - Clean and separated structure
 - Validation is handled automatically
-

Required Libraries

```
<script src="jquery.js"></script>
<script src="jquery.validate.js"></script>
<script src="jquery.validate.unobtrusive.js"></script>
```

How It Works

1. Model contains validation attributes
 2. ASP.NET converts them into HTML attributes
 3. jQuery reads those attributes
 4. Validation runs automatically
-

Example Flow

Model:

```
[Required]  
public string Email { get; set; }
```

Generated HTML:

```
<input data-val="true" data-val-required="The Email field is required" />
```

JavaScript reads `data-val` attributes and validates

Benefits

- No need to manually write JavaScript
 - Clean code
 - Works automatically with ASP.NET Core
-

Example:

Registration Form:

- Name → Required
- Email → Must be valid
- Password → Minimum length
- Confirm Password → Must match

Validation ensures:

- No empty fields
 - Proper format
 - Correct matching
-

QUICK SUMMARY

- Validation ensures **correct and safe input**
- Two main types:
 - Client-side (fast, browser-based)
 - Server-side (secure, backend)
- Data Annotations define rules
- Unobtrusive validation uses `jQuery` automatically